

[DHS/2024/Q2]

2 Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

You are trying to create your own web browser. Two key features of all web browsers are the back button, which allows users to go back to the previous web page, and the history page, which shows all the visited web pages in time sequence and allows the user to revisit any of the web pages directly.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 2.1
        Program code
```

Output:

Task 2.1

You decided to use the Stack Data Structure to create the back button that keeps track of the last 15 web pages visited.

The class `Stack` contains three attributes:

- `stackList`, a 1-dimensional list of web page links (web page link is stored as a string).
- `maxSize`, the maximum number of web page links that can be stored in the stack.
- `topPointer`, the index of the top web page link in the stack, initialised to -1.

The class `Stack` has the following methods:

- a constructor that initialises `maxSize` and `topPointer` to appropriate values for an empty stack. It also initialises the correct number of `None` elements in `stackList` based on the `maxSize`.
- `push(webpage_link)` that takes a web page link(string) as a parameter and inserts it to the top of the stack. If the stack is full, the bottom web page link is deleted to make space for the latest web page link to be inserted at the top of the stack.
- `pop()` that returns the top web page link and removes it from the stack. Returns "History is empty" if the stack is empty.
- `display()` shows the outputs of all the current web page links in the stack, from the top to the bottom of the stack, in the format shown below, without removing any of the web page links. Below shows a sample display when there are three web page links in the stack. Display "History is empty" if the stack is empty.

```
http://ameblo.jp/at/nibh/in/hac/habitasse/platea.jsp<- topPointer
http://lycos.com/ac/est.png
http://tinyurl.com/phasellus/sit/amet/erat/nulla/tempus.jsp
```

Write program code to declare the class `Stack` with all its attributes and methods. [9]

Task 2.2

In a new cell, write program code to: [2]

- Initialise a new `Stack`.
- Read the data in the file `Websites.txt` and push each web page link into the stack.
- Call `display()` method with each insertion of a web page link.
- Call `pop()` 16 times and call `display()` with each `pop()`.

[DHS/2024/Q2 Cont ']

Task 2.3

You decided to use the Linked List Data Structure to create the history page.

Write the `LinkedList` class in Python. Include the following methods:

- `insert(webpage_link)` creates a node that stores the web page link(string) and inserts the node at the beginning (head) of the linked list.
 - `delete(webpage_link)` attempts to delete the node that stores the given web page link value. If the web page link is not found, return `None`. Else return the deleted node.
 - `revisit(webpage_link)` moves the node that stores the given web page link value to the beginning (head) of the linked list. Display `Page Not Found` if the web page link is not found.
- `display()` shows the outputs of all the current web pages in the linked list, from the first node to the last node in the linked list, in the format shown below, without removing any of the web page links. Below shows a sample display when there are three web page links in the linked list. Display "History is empty" if the linked list is empty.

```
http://ameblo.jp/at/nibh/in/hac/habitasse/platea.jsp
http://lycos.com/ac/est.png
http://tinyurl.com/phasellus/sit/amet/erat/nulla/tempus.jsp
```

Write program code to declare the class `LinkedList` with all its attributes and methods. [9]

Come up with appropriate code to test that your `revisit()` method in Linked List is working correctly. You may reuse the web page links in `Websites.txt` in your tests. [2]